Handoff 09: Teaching design and lesson feedback (stage 9, documentation only)

Date: 2026-10-26 (America/Toronto)
Written by: Claude Code (implementer), overnight session
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. **Documentation only**: no application code, content, checker, XP, settings or deployment changed.

> **How to use this file:** upload it with `build.md`, `docs/plan/backlog.md`, `docs/plan/course-catalog.md` and the three new files in `docs/plan/` (`teaching-design.md`, `lesson-feedback.md`, `ai-helper.md`). The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 9, which is documentation only:
a teaching design grounded in programming-education research, a
specification for a lesson feedback engine, a deferred design note for
an optional AI "explain my error" helper, new backlog items, and an
investigation of why print("hi") then print("woah") passes the first
unit 1 challenge. I'm attaching build.md, the backlog, the course
catalog, the three new plan files and this handoff. Check that the
research is described accurately and not overclaimed, that every rule
can be tested, that no feedback path can hand a learner a full
solution, that the AI helper design is safe for students aged 13 to 17,
and that the backlog IDs follow build.md. Explain at a first-year
computer engineering level.
```

1. Where we are

Stage	What	Status
1–6	Content, ledgers, catalog, goals, backlog, rate limits and headers	Verified live
7	SEC-03 server-side sessions	Merged (PR #13); your live check not yet recorded
8	SEC-09 nonce-based CSP	PR #14, open; this PR does **not** depend on it

Stage	What	Status
9	Teaching design and lesson feedback	**in_review**: this PR

2. Handoff record (build.md template)

Field	Value
Task	Investigate the unit 1 "Make it your own" check (read-only); write `teaching-design.md`, `lesson-feedback.md` and `ai-helper.md`; add backlog items
Human goal	Agree on how we teach and how feedback works before writing unit 2, and fix an honesty gap in unit 1 first
Relevant build.md sections	3 (success), 4 (teaching contract), 5 (curriculum), 8 (developer tools), 12 (quality gates), 13 (backlog), 14 (AI collaboration)
Repository and branch	`torontojohn/python-academy`, branch `docs/teaching-design` (assumption 1)
Base commit	`4848d49` (`main` after PR #13)
Implementer / reviewer	Claude Code / owner, plus ChatGPT or Claude chat
Deployment status	Not applicable (documentation only); not merged
New dependencies	None

Files changed

File	Change
`docs/plan/teaching-design.md`	New. Principles with their evidence and how we apply them; task types; how support is removed gradually; spaced review; per-course sections for Python, Hackathon Ready, Git and GitHub, web, SQL and a new Working with AI course; video script rules; review process; measures; references
`docs/plan/lesson-feedback.md`	New. The feedback engine specification: error translations, line highlighting, check-specific messages, hints beside the error, predict-first, the "never a full solution" rules and their tests. Appendix A is the unit 1 investigation
`docs/plan/ai-helper.md`	New. Design note only, deferred: server-side key, off by default, per-user and global daily caps, hard provider spending limit, no grading, privacy, ages 13 to 17, decisions needed before building
`docs/plan/backlog.md`	New items PLAN-03, ENG-04, ASSESS-07, ENG-05 (deferred), TOOL-03, EDU-10; EDU-04 (unit 2) now depends on ENG-04 and ASSESS-07; suggested order updated
`docs/plan/course-catalog.md`	Proposed `ai` course added to the catalog table (section 3.3); session record

File	Change
`build.md`	Section 2 lists the three new documents

3. Unit 1 investigation: why `print("hi")` then `print("woah")` passes

Details and the full table are in `lesson-feedback.md` appendix A. In short:

- **Which card.** Every unit 1 lesson ends with a card headed "Make it your own". This code passes only the **first** lesson's challenge, `hello` ("Open the story"). It fails the other four, whose checks test what their wording promises.
- **What the `hello` check tests:** the code produces exactly two printed lines, neither empty, with exactly two top-level statements. Nothing else.
- **What it promises:** the prompt asks for "a location, then a surprise", and the success message tells every passing learner **"Your scene has a setting and a surprise"**.
- **What also passes,** all run read-only through the real checker:
- the same line twice
- `print(".")` twice
- `print(1)` and `print(2)`
- the lesson's worked example, copied unchanged
- hint 3, copied unchanged
- **Verdict.** The check does what the prompt's second sentence says (two non-empty messages). It does **not** do what the first sentence or the success message say, so the message claims qualities it never tested. That breaks `build.md` sections 3 and 12. Hint 3 is also a complete passing answer.
- **Also found:** `input` hints 1 and 2 together, and `variables` hints 1 and 3 together, are complete passing answers. All three cases were run through the checker.
- **Proposal** (ASSESS-07, not done here):
- Reword the success message to say only what was checked, plus a self-check question.
- Require the two lines to differ, contain letters, and not be the example or a hint copied unchanged.
- Reword the prompt to match.
- Replace hint 3 with a partial pattern.
- Do **not** try to detect "location" or "surprise" automatically.
- Under the proposal, `hi`/`woah` still passes **deliberately**, and the message says honestly what was checked.

Not changed: the check, the content, the tests. The task said "do not change the check yet".

4. Summary of the three documents

- `teaching-design.md`**:**
- Principles: worked examples and fading, predict-then-run (PRIMM), use-modify-create, fill-in-the-gap and Parsons problems, fix-the-bug, subgoal labels, retrieval and spaced review, small projects,

feedback, honest checks.
- Each principle gives its evidence and how we apply it.
- Seven proposed task types, then per-course methods and task kinds.
- The Working with AI course has seven proposed units.
- Research on rewritten error messages is stated as **mixed**, not as a sure win.
- `lesson-feedback.md`:
- A translation table of 19 common errors, with real Python 3.12 messages captured locally.
- Line highlighting without a new dependency, and accessible: a symbol and text, not colour alone.
- Check rules with their own messages, and pass messages that claim only what was verified.
- A three-level hint ladder with no level 4 and no "show solution".
- Predict-first on code cards.
- Five rules against revealing a full solution, each with a test.
- `ai-helper.md`:
- Deferred and off by default; it never grades, never writes the program, and has no chat.
- The key exists only on the server.
- Database-backed caps per user and in total, and a hard spending limit at the provider.
- Minimal data is sent, and nothing is stored by us.
- A parent switch plus the student pressing the button each time.
- Provider terms for 13 to 17 year olds must be checked first.
- Ten decisions to make before building.

5. New backlog items

IDs follow `build.md` prefixes; every new number was checked as unused.

ID	Priority	Item	Status
PLAN-03	Next	Review the teaching design	proposed
ENG-04	Next	Lesson feedback engine. **Before unit 2**	proposed
ASSESS-07	Next	Make the unit 1 `hello` check honest, and fix the hints that add up to answers. **Before unit 2**	proposed
ENG-05	Later	Optional AI helper ("Explain my error")	deferred
TOOL-03	Later	Working with AI course (`ai`)	proposed
EDU-10	Later	Video scripts for lessons	proposed

EDU-04 (unit 2) now lists ENG-04 and ASSESS-07 as dependencies, and the suggested order puts them before it.

6. Assumptions (decisions I took a default for)

1. **Branch name.** The task did not name a branch, so I used `docs/teaching-design`.

2. **IDs.** - The feedback engine is ENG-04 and the AI helper ENG-05 (engineering work). - The unit 1 fix is ASSESS-07 (assessment). - The course is TOOL-03, matching the "Working with AI" tutorial in `build.md` section 8. - Video scripts are EDU-10, which was unused. - The course ID is `ai`.
3. **"Make it your own"** is the heading of every unit 1 challenge card. I took your example to mean the first lesson's card, the only one that code passes, and checked all five.
4. **No stage row** was added to the backlog's "Where we are" table. PR #14 adds a row in the same place, and adding one here would create a merge conflict between two PRs that must not depend on each other. Add "9. Teaching design and lesson feedback" when the second of them merges.
5. **Word minimum for unit 1.** Default: **no** minimum number of words per line (`lesson-feedback.md` section 11, decision 5).
6. **References** were cited from memory of the published literature. I did not re-read them in this session, and both documents say so. Check each one before quoting it to learners.

7. Acceptance criteria

- Unit 1 investigation done read-only, with a verdict and a proposal, and the check unchanged. **Met** (section 3).
- `teaching-design.md` covers worked examples, predict-then-run, fix-the-bug, fill-in-the-gap, gradually removing support, spaced review and small projects, with per-course sections for Python, Hackathon Ready, Git and GitHub, web, SQL and Working with AI (setting up Claude, Claude Code, GitHub, connecting tools), each with method and task kinds. **Met.**
- `lesson-feedback.md` specifies error translations, line highlighting, check-specific messages, progressive hints beside the error, predict-first tasks, and how feedback never reveals a full solution. **Met.**
- `ai-helper.md` is a design note only, covering a server-side key, per-user daily caps, a hard provider spending limit, no grading, off by default, privacy and age considerations for 13 and over, and decisions before building. **Met.** Nothing was built.
- Backlog items with `build.md`-style IDs; the feedback engine and the unit 1 fix come before unit 2. **Met.**
- One PR, documentation only, all tests run before opening it. **Met.**

8. Tests run (exact commands and results)

Nothing executable changed, so these confirm that `main` plus this PR is still green. Run from the repository root on macOS (Python 3.12.14 in `backend/.venv`, Node 24):

Command	Result
`cd backend && .venv/bin/python -m pytest -q`	`177 passed, 9 skipped` (the 9 need PostgreSQL)
Same with `TEST_POSTGRES_URL` set to a local throwaway PostgreSQL 16	`186 passed`

Command	Result
`cd frontend && npx tsc --noEmit`	No errors
`cd frontend && npm run build`	Compiled successfully
`cd frontend && CI=1 npm run test:e2e`	`27 passed` (PR #14's three extra tests are not on `main`)
Relative links in the three new documents	All resolve
The investigation (read-only): `backend/.venv/bin/python` calling `app.lesson_checks.assess` on the cases in section 3	Results as reported in `lesson-feedback.md` appendix A
Error messages for the translation table: the same Python 3.12 running each broken snippet	Messages as quoted in `lesson-feedback.md` section 4

9. Mocks and unverified behaviour

- **No mocks were used for the investigation.** It called the real checker function directly, not through the API or the browser.
- **The error messages were captured with local CPython 3.12**, not inside Pyodide. Pyodide 0.27 uses Python 3.12, so the last line of each message should match. The full traceback text Pyodide returns, including its internal frames, was **not** captured. `lesson-feedback.md` section 10 makes capturing it the first step of ENG-04.
- **Research claims** are from memory, as stated in assumption 6.
- **Provider terms** (minimum ages, data use, prices) were deliberately **not** asserted. `ai-helper.md` and `teaching-design.md` section 6.6 say they must be checked at decision time.
- The Playwright tests mock the API, as before; nothing in them changed.

10. Questions for the owner

1. **Unit 1 wording and checks** (`lesson-feedback.md` appendix A): should I build the proposal as written under ASSESS-07, and with no minimum word count (assumption 5)?
2. **Hints:** may I also rewrite the `input` and `variables` hints that add up to full answers, in the same ASSESS-07 PR?
3. **Editor:** keep the plain text box with a highlight overlay for ENG-04 (no new dependency, proposed), or plan a code editor library for unit 2?
4. **Working with AI course:** for the setup units, which approach do you prefer for students under 18: parent-led setup, an adults-only label, or walkthroughs only? This depends on each provider's terms.
5. **AI helper:** is deferring it, with no spending, the right default (proposed)?

11. Live checks for you

Nothing is deployed by this PR, so there is nothing to check on the live site afterwards. To see the problem for yourself, before ASSESS-07:

- [ ] **Reproduce the unit 1 finding on the live site** (no sign-in needed). Open `https://python-academy-dji8.onrender.com/preview?lesson=hello&card=5`. Type `print("hi")`, press Enter, then type `print("hi")` again (the same line twice). Press **Run & check challenge**. It passes and says "Your scene has a setting and a surprise." That is the honesty gap.
- [ ] **Hint 3 is an answer.** On the same card, show all three hints. Copy the two lines from hint 3 into the editor and check: it passes.
- [ ] **The other lessons are stricter.** Open `/preview?lesson=variables&card=5`, enter `print("hi")` and `print("woah")`, and check: it asks you to use the `hero` and `tool` variables.
- [ ] **Read and mark up the documents:** `docs/plan/teaching-design.md` sections 2 to 6, `docs/plan/lesson-feedback.md` sections 4, 7, 8 and appendix A, and `docs/plan/ai-helper.md` section 9. Note anything you disagree with; that is PLAN-03.
- [ ] **Answer section 10**, so ASSESS-07 and ENG-04 can start.

12. Next concrete action

The owner reviews this PR and answers section 10. Then, in separate PRs: ASSESS-07 (the unit 1 fix with its tests), then ENG-04 (the feedback engine, unit 1 first), then ENG-03 and unit 2 (EDU-04). Meanwhile PR #14 (SEC-09) and the stage 7 live check are still waiting for you.